Smart Campus Agent System

Multi-agent AI platform for campus operations · Sujan S · interview / viva notes · page 1 of 3 — the pitch & the stack

↓ say this first (30 seconds)

A supervisor LLM routes a plain-English request to a **specialist agent**, which calls **typed Python tools** — never raw SQL. Hard scheduling is done by an OR-Tools CP-SAT solver, not the LLM, and any decision that affects a real person **pauses for human approval** using LangGraph `interrupt()`. So: the LLM *proposes*, deterministic code *disposes*.

1 · The problem it solves

- Building a college timetable by hand = **days of work**, clashes, unfair teacher loads.
- One teacher goes on leave → HOD scrambles for a substitute, and students **lose that subject's hours** permanently.
- The rules ("no labs after lunch", "Dr. Rao is free only on Tue") live in **people's heads**, not in any system.

2 · What it actually does

F1 · Timetable Agent done	Generates a **clash-free week** for every section from a stored rulebook. Grid UI + PDF download.
F2 · Leave & Substitution flagship, done	Leave approved → agent builds a **period-exchange** plan → HOD approves → everyone notified.
2.4 Constraint rulebook done	Every scheduling rule is a DB row with a validator + an English rendering.
2.5 Timetable Assistant done	One English sentence → a whole ordered plan of typed operations, applied in one transaction.
F3 Events · F4 RAG planned	Event/room booking + retrieval over campus documents (ChromaDB) + WebSocket notifications.

3 · Tech stack — and why each piece

Layer	Choice	Why I picked it
Frontend	**Next.js 14 + TypeScript + Tailwind**	App-router dashboard, glassmorphic UI, live agent-trace sidebar.
Backend API	**FastAPI + Uvicorn** (Python 3.11)	Async-native, auto-generated OpenAPI/Swagger, first-class WebSockets.
Agent orchestration	**LangGraph ≥ 0.2**	Cyclic graphs + typed shared state + **checkpointers** + `interrupt()`. A plain chain can't pause for a human and resume later.
LLM	**Provider-agnostic layer** Gemini · Claude · OpenAI · Ollama	One factory `get_llm()` using `init_chat_model`; provider swapped by **.env only**. No vendor lock-in; Ollama = offline demo backup.
Scheduling engine	**Google OR-Tools (CP-SAT)**	Constraint satisfaction is a **solver's** job. It *proves* feasibility or tells you *why* it's infeasible — an LLM can only guess.
Database / ORM	**SQLAlchemy 2.x** → SQLite (dev), Postgres (prod)	**20 tables.** Swapping DB = one connection string. Idempotent seeding on boot.
Auth	**JWT** + role-based access	Roles: **admin / faculty / student** — drives what each user can see & approve.
Background jobs	**APScheduler**	In-process "safety sweep" cron — catches leaves that slipped through. No Redis/Celery at this scale.
Durability	**LangGraph SQLite checkpointer**	One `thread_id` per conversation → a paused workflow survives a server restart.

4 · Request lifecycle — draw this on the whiteboard

Browser (Next.js) ⟶ **FastAPI `/api/agent`** JWT checked
- ⟶ **LangGraph StateGraph** (loads checkpoint for this `thread_id`)
  - ⟶ **Supervisor node** — LLM classifies intent (keyword fallback if no API key)
    - ⟶ conditional edge → **timetable** | **substitution** | **general** specialist
      - ⟶ **tools/*.py** (typed) → SQLAlchemy / CP-SAT solver → DB
        - ⟶ result + **trace** streamed back to the sidebar → END

Key nuance: system-sourced triggers (a leave approval, the scheduler sweep) **skip the LLM entirely** and route deterministically — machine-to-machine events shouldn't pay for, or risk, a model call.

The technical core

5 · LangGraph & the supervisor pattern

- A **StateGraph** over a typed **AgentState** (messages, intent, trace, payload) — every node reads and returns a **partial state update**, so state flow is explicit, not hidden in closures.
- **Supervisor node** = router. The LLM classifies intent; a conditional edge fans out to a specialist node. Adding a capability = add a node + register it — no keyword lists to maintain.
- **Graceful degradation:** missing/invalid LLM key → deterministic keyword routing. The demo can never fail because of a network.
- **Checkpointer** (SQLite): state persisted per **thread_id** after every node, so a workflow is **durable and resumable**.
- **Human-in-the-loop:** **interrupt()** freezes the graph *mid-execution* and returns to the caller. The HOD sees an approval card hours later; the API resumes with **Command(resume=…)** and the graph continues from that exact node, not from the beginning.

6 · Why a solver, not an LLM — the strongest answer I have

An LLM cannot **guarantee** that no teacher is in two rooms at once. CP-SAT either returns a solution that **provably** satisfies every hard constraint, or reports **infeasible with a reason**. Correctness here is a maths problem, so I gave it to a maths engine and kept the LLM for language.

The model (app/solver/timetable_model.py)

- **Decision variables:** boolean **x[lesson, day, period, room, teacher]** — "this lesson of this section happens in this slot".
- **12 hard constraint groups (H1–H12)** — no teacher clash · no room clash · no section clash · exact weekly quota per subject · labs as contiguous double periods · room-type must match subject · teacher availability windows · max periods per teacher per day · first/last-period bans · elective band reservation · pinned teaching assignments · one lesson per section per slot.
- **Weighted soft objective:** teacher-load **fairness** + **preferred periods** + minimise **mid-day gaps** for both students and staff. Hard = must; soft = the solver's taste.

★ Incremental re-planning — the detail interviewers like

Naively, adding one subject re-solves from scratch and **rewrites the whole week** — unusable for a real college. Instead the published timetable is fed back in as **solution hints**, plus a **move penalty** on any lesson that changes position.
Result: adding a subject moves **0** existing lessons and solves in **~0.6 s**, versus **~8 s** and a completely rewritten week for a cold **?fresh=true** re-plan.

7 · The constraint registry (Phase 2.4)

- The **constraints** table is the single source of truth. **POST /api/timetable/generate** takes **no request body at all** — it compiles the table into solver options via **compile_options()**.
- **app/tools/constraints.py** holds a **CATALOG of 14 rule kinds**. Each kind declares three things:
  ① allowed **scopes** ② a **validator** against real DB entities ③ an **English renderer**
- The admin form **and** the LLM's prompt are both generated from that catalog — so adding a new rule kind means editing the catalog and the solver, and the UI + the AI update themselves.
- **Scope resolution: section > semester > global.** One solve therefore produces a differently-shaped week per semester, from one shared rulebook.

8 · The layering rule I'd defend in any review

Agents never touch the database. Agents call **tools**; tools own SQLAlchemy and the solver. So every capability is unit-testable without an LLM, the LLM can be swapped or removed, and the same tool serves the chat agent, the REST API and the scheduler.

Backend layout: **api/** routers · **agents/** graph + supervisor + specialists · **tools/** timetable, constraints, ops, exchange · **solver/** CP-SAT model · **db/** models, seed, session (with additive schema migration) · **core/** llm, config, security.

9 · The 20 tables, grouped + numbers worth quoting

Academic: User · Teacher · Subject · Section · Room · TimeSlot | **Timetable:** TimetableEntry (the solved grid) · TimetableConfig · TeachingAssignment (pins *section+subject → teacher*)
Rules: Constraint | **Electives:** ElectiveGroup + ElectiveOffering (sem 6/7 bands) | **Leave:** Leave · PeriodExchange · Approval · Notification | **Phase 3:** Event · Booking · Document

Quotable: **20** tables · **12** hard constraint groups · **14** rule kinds · **23** typed ops · **4** LLM providers · **0.6 s** warm vs **8 s** cold re-solve.

Flagship flow · NL assistant · Q&A

11 · F2 — Leave → period exchange (walk through this slowly)

1. Faculty applies for leave on `/leaves`.
2. Admin approves → **system trigger** (deterministic, no LLM) → Substitution Agent wakes.
3. The tool builds a **period exchange**, not a substitution: a partner teacher of the **same section** swaps *their* lesson into the empty slot, and the absent teacher **recovers it later** in the partner's slot.
   Subject hours are preserved and the published timetable is never rewritten.
4. Candidates are **scored**: any swap creating same-subject adjacency for students, or 3+ consecutive teaching periods for a teacher, is **penalised** and carries a ⚠ warning on the card.
5. `interrupt()` → HOD approval card on `/approvals` → resume with `Command(resume=…)`.
6. Notifications to the partner + the returning teacher; the day is served as a **dated overlay** — `/api/timetable/effective/{section}?date=`.

The line to land: a naive system finds a warm body to cover the class; mine keeps the **curriculum contract** intact — the students still get all their hours of that subject.

12 · Phase 2.5 — Timetable Assistant (English → plan)

- `POST /api/assistant/interpret`: one sentence → a whole **ordered plan**, which will **create whatever is missing** — teachers, subjects, rooms, classes, teacher–subject maps, pins, elective bands, rules, and the regeneration itself.
- **23 typed operations** in `app/tools/ops.py`; every op implements **validate / describe / apply**. The LLM can only emit ops that exist → a closed vocabulary is my anti-hallucination guard.
- **Forward references:** a rule may scope to a class the *same plan* is about to create. Pending entities are stubbed into the validator's context with **negative ids**, then **re-validated for real** at apply time.
- **Bulk forms** `{defaults, items}` keep big requests fast — **output** tokens are what the admin actually waits on.
- The plan is **previewed in plain English** for approval, then applied as **one transaction**: all-or-nothing.

13 · Questions they will ask — and my answers

- **Q. Why LangGraph and not just function calling?**
  Cyclic graphs, a typed shared state, built-in checkpointing, and `interrupt()`. A linear chain cannot pause mid-flow for a human approval and resume from the same node hours later — mine can, because state is checkpointed per thread.
- **Q. What happens when the LLM hallucinates or the API is down?**
  Nothing the model says reaches the database unchecked. It can only emit ops from a closed registry; each is validated against real DB entities, rendered in English for approval, and applied transactionally. No key at all → deterministic keyword routing still works.
- **Q. How do you know the generated timetable is correct?**
  The clash rules are **hard** constraints in CP-SAT, so a returned solution is a mathematical proof they hold; and the saved grid is re-verified. If it's impossible, the solver names the constraint that made it impossible instead of silently producing garbage.
- **Q. Would this scale to a real campus?**
  SQLite → Postgres is a connection-string change through SQLAlchemy. The solver is the bottleneck, and the warm-start hints keep normal edits sub-second. APScheduler → Celery if jobs ever outgrow one process.
- **Q. Hardest part?**
  Making **regeneration stable** — my first version re-solved from scratch and shuffled everyone's week for one added subject; hints + a move penalty fixed it. And supporting **forward references** in a plan, which needed the two-phase validation with stub ids.
- **Q. What's next?**
  Phase 3: event/room booking, retrieval-augmented Q&A over campus documents with ChromaDB, and WebSocket push for live notifications.

↓ closing line

"The interesting part of this project isn't that it calls an LLM — it's **where I chose not to**. Language goes to the model; correctness goes to a solver; consequences go to a human."